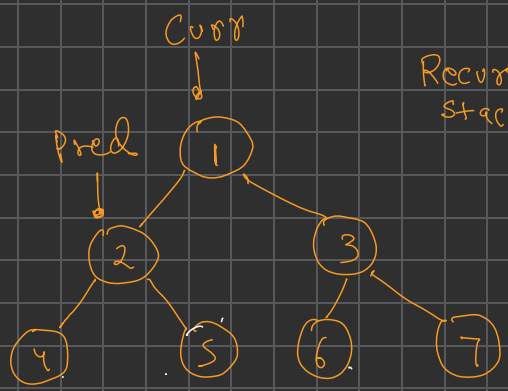
inorder

LNR



Recursion stack : Space com $O(h) \Rightarrow O(n)$

$O(1)$

Space

Morris traversal

4 2 5 1 6 3 7 ⇒ element

4 2 5 1 6 3 7

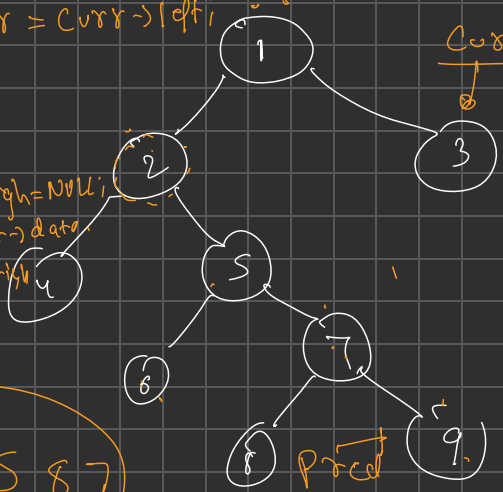
inorder

1. Node pe ho
2. if (Left exist) left side ke right most node pe jaye → mere upar link
3. if (Left exist) Node right

```
Node * curr = root;

while (curr)
{
    if (!curr -> left)
    {
        cout << curr -> data;
        curr = curr -> right;
    }
    else
    {
```

```
Node* pred = curr->left;
while (pred->right && pred->right != curr)
{   pred = pred->right;
}
if (!pred->right)
{
 pred->right=curr;
 curr = curr->left;
}
else
{
 pred->righ=NULL;
 Cout<< curr->data;
 curr=curr->right
}
```



Curr

L N R

① if Left doesn't exist
→ print
→ Right

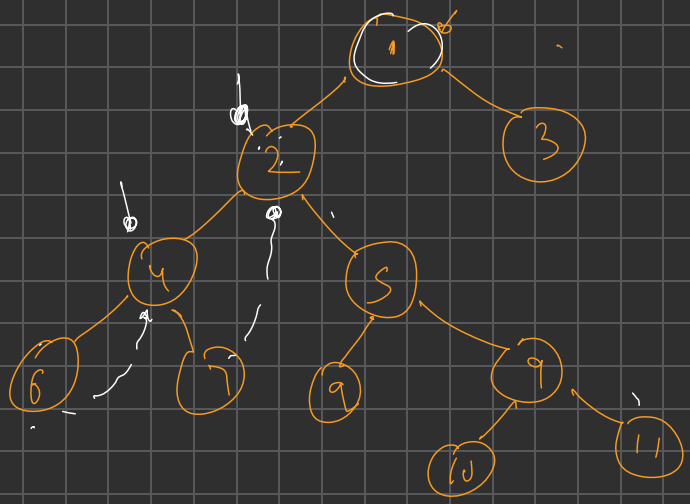
② else
{
left side pe jaungi
→ Right most node pe chlata jaun
→ if u get null
⇒ Break the link
print elem
move to right

1 4 2 6 5 8 7 9 1 3

L N R

in

1



$\Theta(n)$

$\Theta(n)$

1 ⇒ Link

2 Print

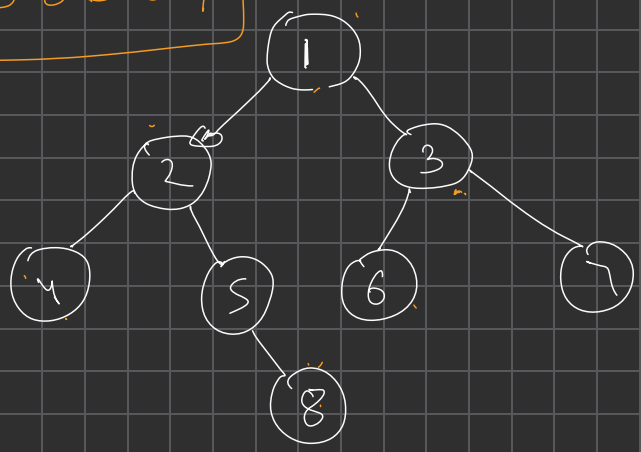
3 Link Tody

1 2 4 5 8 3 6 7 → print

N L R curr

① First time usko tabhi print kar

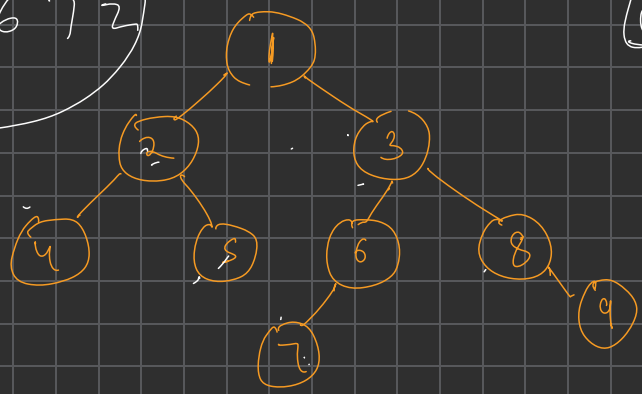
②



1 2 4 5 8 3 6 7

Post order:

1 3 8 9 6 7 2 5 4



LRN

NLR

Pre

Post:

NRL

reverse

4 5 2 7 6 9 8 3 1

1 3 8 9 6 7 2 5 4

① inorder:

~~2~~	~~5~~	~~7~~	~~4~~	3

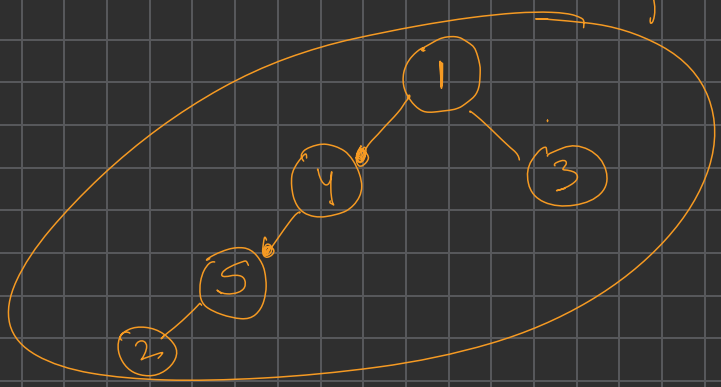
LNR

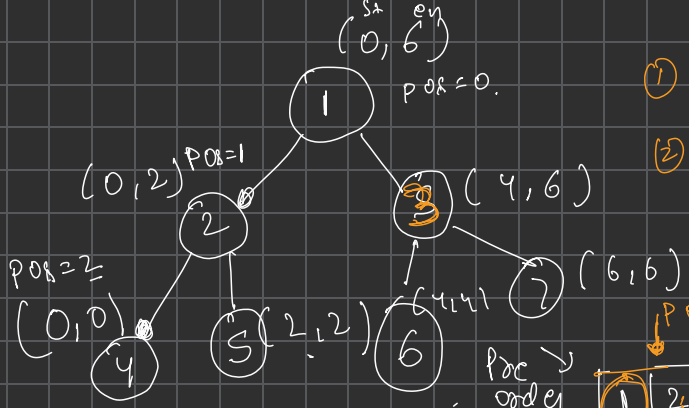
② Preorder:

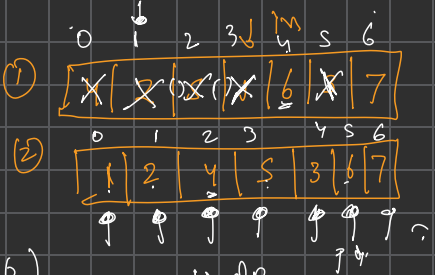
1	4	5	2	3

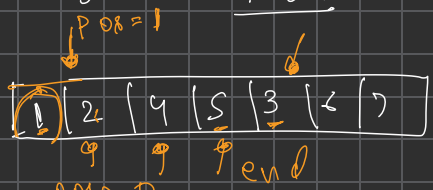
Pre-ord

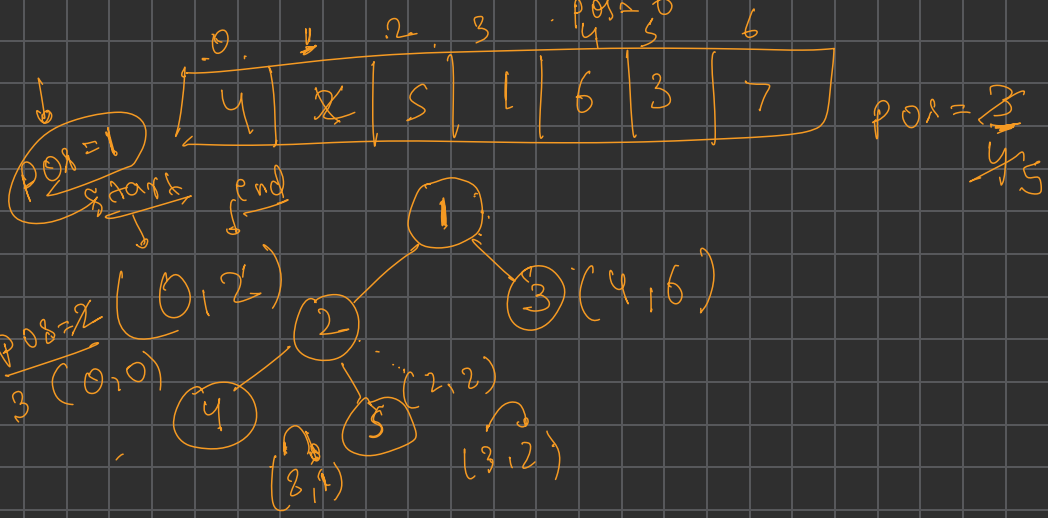
NLR

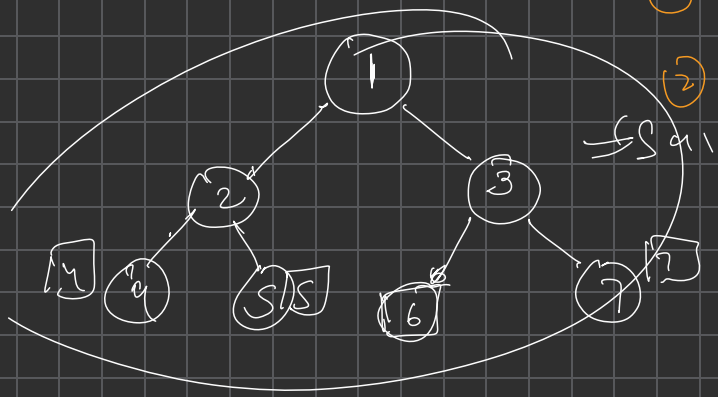



st
en
(0, 6)
pos = 0.
1
(0, 2)
pos=1
2
3
(4, 6)
pos=2
(0, 0)
4
5
(2, 2)
6
(4, 4)
7
(6, 6)
Pre order


0 1 2 3 4 5 6
1
2
3
4
5
6
7
node


pos = 1
1 2 4 5 3 6 7
end


start
0 1 2 3 4 5 6
4 5 1 6 3 7
pos = 0
pos = 1
start
end
pos = 2
(0, 2)
3 (0, 0)
(2, 2)
(3, 1)
(3, 2)
(4, 6)
pos = 3
4, 5



inorder

(1) | 4 | 2 | 5 | ~~1~~ | 6 | ~~3~~ | 7 |

(2) | 4 | 5 | 2 | 6 | 7 | 3 | 1 |

pOIA

L R N